

Git и GitHub: Контроль версий в разработке ПО

Инструменты, без которых невозможна современная командная разработка программного обеспечения.



git



Что такое Git?



**Распределённая
система
контроля
версий**

Отслеживает
изменения кода и
полную историю
разработки

**Создан в
2005 году**

Линус Торвальдс
разработал Git для
ядра Linux

**Параллельная
работа
без
конфликтов**

Каждый
разработчик имеет
полную копию
репозитория

Основные функции Git

Коммиты

Сохранение снимков кода с описанием изменений

Ветки (Branches)

Параллельная разработка функций без влияния на основной код

Слияние (Merge)

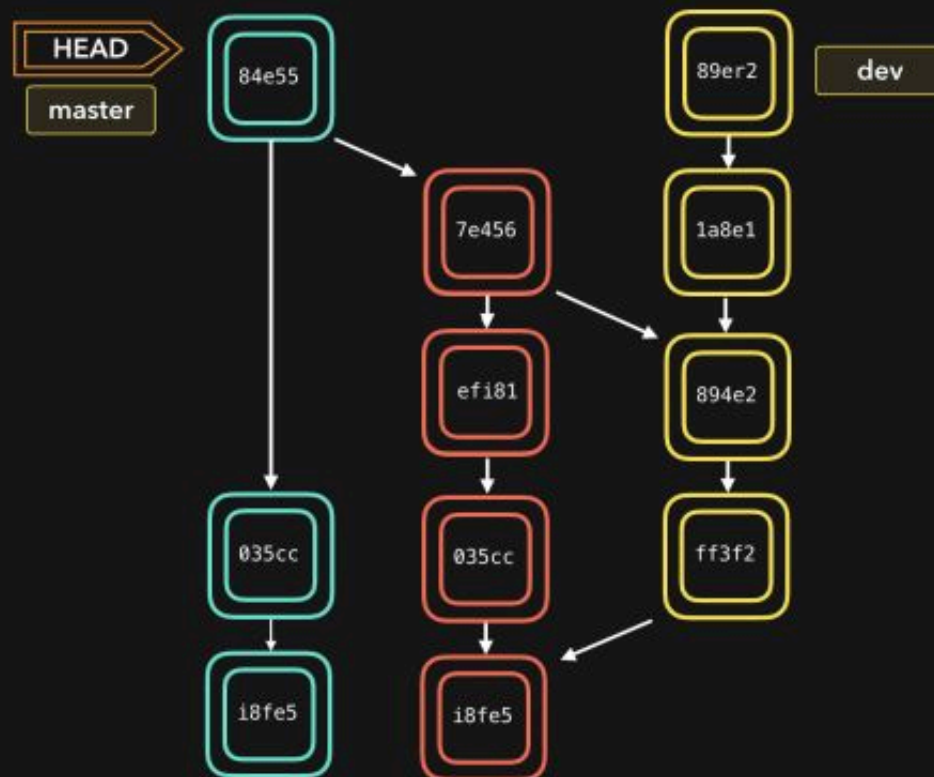
Объединение изменений из разных веток

История версий

Полный журнал всех изменений с возможностью отката

Теги

Маркировка важных версий и релизов



```
bash

master$ git reflog
84e55 HEAD@{0}: commit(merge)
035cc HEAD@{1}: commit
8d83a HEAD@{2}: reset moving to head~1
18fe5 HEAD@{3}: commit(initial)
master$ git -am "Fix 🚀"
master$ git rebase -i i8fe5
```



Git Commands
VISUALIZED

Что такое GitHub?

Облачная платформа для хранения Git-репозитория и совместной работы над проектами любого масштаба.

2008

Год основания

100M+

Разработчиков

2018

Год покупки Microsoft

Поддерживает как открытые (Open Source), так и приватные проекты.

Fork me on GitHub

Ключевые возможности GitHub



Pull Requests

Предложение изменений с возможностью обсуждения и рецензирования



Issues

Отслеживание ошибок, задач и планирование разработки



GitHub Actions

Автоматизация CI/CD: тестирование, сборка, развёртывание



Code Review

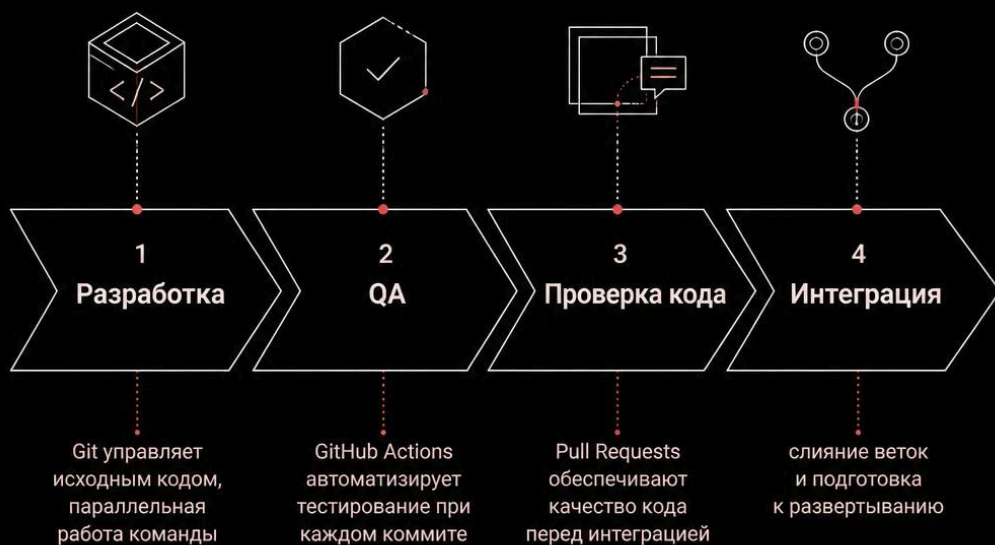
Встроенные инструменты проверки кода перед слиянием



Wikis и Документация

Документирование проектов прямо в репозитории

Git и GitHub в жизненном цикле ПО



Разработка (Development)

Git управляет исходным кодом и обеспечивает параллельную работу всей команды без конфликтов.

Контроль качества (QA)

GitHub Actions автоматически запускает тесты при каждом коммите.

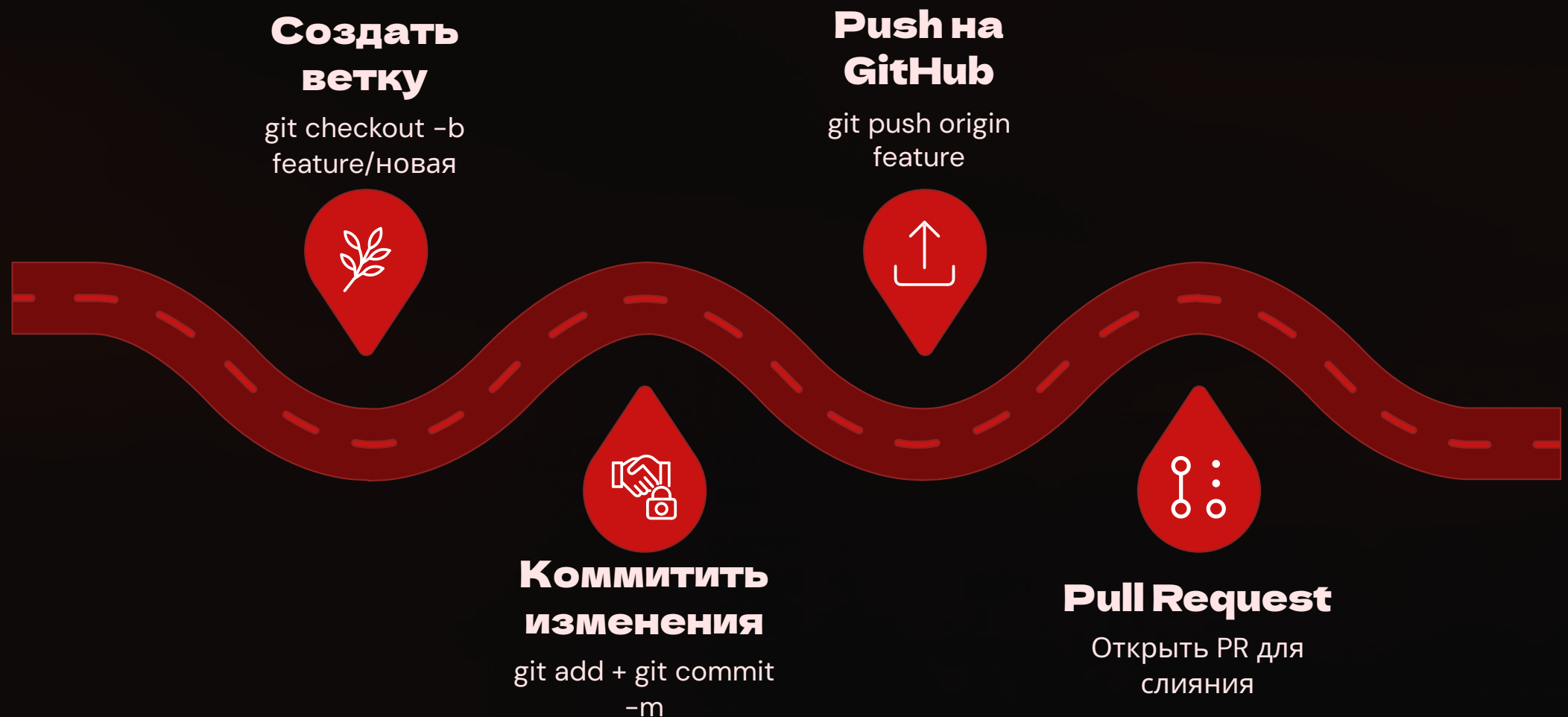
Code Review

Pull Requests обеспечивают проверку кода коллегами перед интеграцией.

Интеграция (Integration)

Слияние веток и подготовка стабильной версии к развёртыванию.

Типичный рабочий процесс



Стандартный цикл разработки фичи от создания ветки до автоматического деплоя через GitHub Actions.

```
carla@studio:~/git-projects/cpputest$ git status
On branch newbranch
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

       deleted:    CppUTest.mak
       modified:   scripts/UnityTemplates/ClassNameCTest.cpp
       modified:   scripts/appveyor_ci_build.ps1

no changes added to commit (use "git add" and/or "git commit -a")
carla@studio:~/git-projects/cpputest$ git add *
carla@studio:~/git-projects/cpputest$ git status
On branch newbranch
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

       modified:   scripts/UnityTemplates/ClassNameCTest.cpp
       modified:   scripts/appveyor_ci_build.ps1

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

       deleted:    CppUTest.mak

carla@studio:~/git-projects/cpputest$ █
```

Преимущества Git и GitHub

Безопасность

Полная история изменений и откат к любой версии

Сотрудничество

Команда работает одновременно без конфликтов

Прозрачность

Видно, кто и когда изменил любую строку кода

Масштабируемость

От одного разработчика до тысяч участников

Интеграция

CI/CD, мониторинг и инструменты развёртывания из коробки

Заключение

Стандарт индустрии

Git и GitHub — обязательные инструменты для любой современной команды разработки программного обеспечения.

Критически важны для

- Этапа разработки (Development)
- Контроля качества (QA)
- Автоматизации CI/CD



Эффективное сотрудничество

Команды работают быстрее и качественнее



Качество кода

Code Review и автотесты предотвращают ошибки



Необходимость

Незаменимы в любой команде — от стартапа до корпорации